Prerequisite Gate — Async Python (before Document 09)

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Not a numbered document — a short, focused checkpoint. LangGraph's way of running things (streaming, running steps at the same time, `ainvoke` / `astream`) assumes you're comfortable with `async` / `await`. This is the first place in the curriculum where you actually need it, so it's taught here instead of earlier, where it would have no real use case yet.

Prerequisites

08_rag

How to Read & Practice This Gate

- **What:** running several things at once in Python — `async` / `await`, the event loop, `asyncio.gather`.
- **Why:** LangGraph, and any real production API, depend on this. Skip it, and Doc09 will feel like magic instead of something you understand.
- **When:** any time you're making several separate calls (API calls, database queries) that don't depend on each other's answers.
- **How to practice:** 1. Read the Real Python walkthrough in full — this idea really does need the longer explanation, not just the short official-docs version. 2. Do the **Basic** exercise closed-book. 3. Do the **Intermediate/Real-world** timing comparison — the actual numbers, not just the theory, are what make this click. 4. Do the **Failure** exercise on purpose — breaking things at running-together-ness on purpose teaches this faster than just reading about it. 5. Use **Hint 1 → Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, explain out loud why async helps for waiting-on-something work, and does nothing for heavy-computing work. If you can't, you're not done.

Jump to: Core Concepts · Practice Exercises

The Story — what this document is actually building

Picture this: Doc09's LangGraph agent needs to call three tools to answer one question — maybe a search, a lookup, and a calculation. Nothing about those three calls depends on the other two's answers. Run them one after another with plain, blocking Python, and you wait for all three, back to back — a 2-second call, another 2-second call, another 2-second call, six seconds total, even though the computer was just sitting idle waiting for a network reply the whole time.

That idle waiting time is the whole story of this gate. `async` / `await` doesn't make Python faster at computing — it lets Python use the time it would otherwise spend just waiting on a network reply to go start another task instead. One thread, one event loop, switching between tasks only at the moments one of them is waiting on something. `asyncio.gather` is the tool that actually captures that benefit: instead of `await a(); await b(); await c()` (one after another, six seconds), you write `await asyncio.gather(a(), b(), c())` (all three started at once, roughly two seconds — however long the slowest one takes alone).

None of this is a new problem. Doc02 already taught you that the network is unreliable and slow, and that every call needs a timeout and a plan for failure. Async doesn't change any of that — it just lets you do several of those unreliable, slow calls at once instead of one at a time. The status codes, the timeouts, the JSON-shape checking, the retry discipline: all of it still applies, now just with `await` in front.

This has no Build Task of its own — it's a checkpoint, not a project. The reason it exists as its own short document, right here and not earlier, is that Doc09's LangGraph relies on this directly: streaming, running nodes in parallel, and the async client patterns you'll use from here through the rest of the curriculum all assume you're already comfortable with `async` / `await`. Project 8 — MCPBridge is async throughout too, because the MCP client it's built on is async-only. Get this solid now, and both will read like ordinary code. Skip it, and they'll feel like magic you can't debug.

Core Concepts (read this first — everything you need is here)

Topics on this page: The event loop: one thread, many waiting tasks · Why async helps waiting-on-something work, and does nothing for heavy-computing work · `asyncio.gather`: running things at the same time instead of one after another · Sync vs. async client: same tool, different way of running · Common mistakes, and why they're sneaky

The event loop: one thread, many waiting tasks

A single waiter can serve many tables at once, as long as no table needs the waiter every second. Take table 1's order, walk it to the kitchen, and while the food cooks, go take table 2's order, then table 3's. When table 1's food is ready, come back to it. One waiter, several tables, and nobody waits any longer than they have to — the waiter is never standing still doing nothing. Python's **event loop** is that waiter. It runs on **one thread**, and it keeps a list of tasks. Each task is a **coroutine** — what you get back from calling an `async def` function — and a coroutine can pause at an `await` line, hand control back to the loop, and pick up again later from that exact line once its answer is ready.

The loop only ever switches tasks at an `await`, never in the middle of a line of code, so two tasks can never step on each other's variables at the same instant. That's why async code needs no locks the way multi-threaded code does: nothing runs at the same instant, it just takes turns very fast.

How it really works

- Calling an `async def` function does **not** run it. `fetch_user(42)` alone just builds a coroutine object and hands it back, unstarted — like writing an order ticket and leaving it on the counter. Only `await`, `asyncio.gather(...)`, or `asyncio.create_task(...)` actually sends it to the kitchen.
- `asyncio.run(main())` creates a brand-new event loop, runs `main()` to the end, then closes the loop. It belongs in exactly one place: the very bottom of a script, inside `if __name__ == "__main__":`. Call it a second time, or call it from inside code that's already running on a loop, and Python raises `RuntimeError: asyncio.run() cannot be called from a running event loop`.
- `asyncio.create_task(coro)` schedules a coroutine to start running on the loop right now, without waiting for it — it hands back a `Task` object you `await` later to collect the result. This is how you start two things "at the same time" by hand, one call before the other, instead of always going through `gather`.
- A `Task` the loop knows about only through a weak reference can be garbage-collected before it finishes if nothing of yours keeps a reference to it — a "fire and forget" `create_task()` with nothing holding the result is a real way to silently lose work. Keep the task in a variable (or a set) and `await` it eventually.
- Jupyter notebooks and frameworks like FastAPI already run their own event loop for you. Inside either one, write `await main()` (or just `await` calls) directly — never `asyncio.run()`, since a loop is already running there.

Situation	What to do	Why
Waiting on a network reply, a database, a file	`async def` + `await`	The loop can use the waiting time for another task
A short script making one call, then exiting	Plain sync code	Nothing else to run meanwhile — async only adds lines
Inside an already-async framework (FastAPI `async def` route, LangGraph)	`async def` + `await` inside it	The framework already owns and runs the loop
A Jupyter notebook cell	`await main()` directly	The notebook's own loop is already running — `asyncio.run()` raises

Common mistakes:

- *Mistake:* calling an async function and using the result without `await`. → *Symptom:* no error at all — just a coroutine object where you expected data, or a `RuntimeWarning: coroutine`

`'...' was never awaited` buried in the output. → *Fix:* every call to an `async def` function needs `await` (or `gather` / `create_task`) in front of it.

- *Mistake:* calling `asyncio.run()` from inside a FastAPI route or a Jupyter cell. → *Symptom:* `RuntimeError: asyncio.run() cannot be called from a running event loop`. → *Fix:* use `await` directly — `asyncio.run()` belongs only once, at the very top of a plain script.

Where you'll meet it: Doc09 runs on this loop through `ainvoke` / `astream`, the async versions of `invoke` / `stream`. Doc12 and Project 5 put agents behind FastAPI, which runs every `async def` route on one shared loop. Doc11 has several agents waiting on the model at once — the loop is what lets one process look after all of them. Project 8 — MCPBridge and the other MCP projects are async throughout, for the same reason.

Quick cheat sheet:

- One thread, one loop, many tasks — they switch only at `await` lines.
- `async def` builds a coroutine; nothing runs until something `await`s, `gather`s, or `create_task`s it.
- `asyncio.run(main())` exactly once, at the very bottom of a script — never inside async code.
- In Jupyter: `await main()` directly, no `asyncio.run()`.
- Keep a reference to every `create_task()` result, and `await` it before the program ends.

Why async helps waiting-on-something work, and does nothing for heavy-computing work

A cashier ringing up ten customers can start scanning customer 2's items while customer 1's card is still processing — the wait for the card machine is dead time the cashier can spend elsewhere. Now hand that same cashier a box of ten thousand coins to count by hand. There's no "waiting" to fill with something else — the counting itself is the work, and it takes exactly as long no matter how the cashier's attention is split. That's the whole split behind this topic. **I/O-bound** work spends most of its time waiting for something outside the program — a network reply, a database, a disk. **CPU-bound** work spends its time actually computing — math, parsing, resizing. Async helps the first kind enormously, because the loop can fill the waiting time with other tasks. It does nothing for the second kind, because there's no idle time to fill — the CPU is already fully busy, and the loop can only switch at an `await`, which a long calculation never reaches.

How it really works

- A calculation with no `await` inside it holds the one thread until it finishes, and every other task — including ones that were only waiting on the network — waits behind it. Splitting the calculation into smaller `async def` pieces doesn't help either; the total CPU time is the same, just spread across more function calls.
- The real fix for CPU-bound work is `concurrent.futures.ProcessPoolExecutor` (or `multiprocessing`) — separate operating-system processes, each with its own CPU core, genuinely running at the same time instead of taking turns.

- `await asyncio.to_thread(func, arg)` runs an ordinary blocking function (one with no async version, like `requests.get`) in a background thread, so the event loop stays free while it waits. This fixes an I/O-bound sync call. It does **not** speed up a CPU-bound one — a thread still competes for the same CPU, and Python's own internals mean two CPU-heavy threads don't truly run in parallel either.
- `asyncio.Semaphore(n)` caps how many coroutines can be "inside" a block of code at once — the standard way to fire off thousands of embedding calls or LLM calls without tripping a `429` rate limit, the same failure Doc02's retrying topic teaches you to expect and handle.
- **Logging from several concurrent tasks has its own gotcha.** Because the loop interleaves tasks, log lines from task A and task B can land next to each other in the file even though neither task has actually finished yet — two half-written requests interleaved look like one garbled one unless every line carries its own task or request id. Doc01's `get_logger(name)` gives you a named logger per module; for concurrent async work, add a request id through a `logging.Filter` or a `contextvars.ContextVar` so the formatter can print it on every line — without that, a busy log from several agents running at once is unreadable.

Kind of work	Example	What to do
I/O-bound	LLM calls, HTTP calls, database queries	`async` + `await`, `asyncio.gather`
CPU-bound	Parsing huge PDFs, image resizing, heavy math	`ProcessPoolExecutor` — **not** async
A sync call you can't replace	An old library with no async version	`await asyncio.to_thread(func, ...)`
Many calls to a rate-limited API	Embedding 10,000 chunks	`async` + `asyncio.Semaphore(n)`

Common mistakes:

- *Mistake:* making a CPU-heavy function `async def`, expecting it to run faster "because it's async." → *Symptom:* it runs at the same speed, and worse — it freezes every other task on the loop for however long it takes, so a web server stops answering everyone else during that time. → *Fix:* look for a real `await` on a waiting call inside the function. None there? Keep it sync and run it in a `ProcessPoolExecutor`.
- *Mistake:* several concurrent tasks logging through one shared logger with no per-task identity. → *Symptom:* log lines from different tasks interleave mid-message; working out "what happened at 3:04pm" is impossible because three agents were logging at once with no way to tell them apart. → *Fix:* attach a task or request id through a `Filter` / `ContextVar` and put it in the formatter — the same discipline as Doc01's logging, extended for concurrency.

Where you'll meet it: almost everything in an LLM app is I/O-bound — model calls (Doc04), tool calls (Doc06), vector searches (Doc08). The genuinely CPU-bound parts — splitting and cleaning

big documents before a RAG index, running a local embedding model — need processes, not async. Project 12 wraps a slow sync pipeline in `asyncio.to_thread()` so its MCP server doesn't freeze.

Quick cheat sheet:

- I/O-bound (waiting) → async helps. CPU-bound (computing) → it does not.
- An `async def` with no real `await` inside gives no speed-up, and blocks everyone else.
- CPU-heavy work → `ProcessPoolExecutor`. A sync call you can't avoid → `asyncio.to_thread`.
- Many calls, one rate limit → `asyncio.Semaphore(n)`, the async cousin of Doc02's backoff discipline.
- Concurrent tasks logging together need a per-task id in every line, or the log is unreadable.

`asyncio.gather`: running things at the same time instead of one after another

Three chefs, each cooking one dish, working at the same time finish dinner in however long the slowest dish takes — not the sum of all three. One chef cooking all three dishes in turn takes the sum. `asyncio.gather` is what turns your code from the one-chef version into the three-chef version: it takes several coroutines, starts them all on the event loop right away, and waits until every one of them is done. `await a(); await b(); await c()` runs one after another — total time is roughly the sum of all three. `await asyncio.gather(a(), b(), c())` runs them together — total time is roughly the slowest one alone. For three independent 2-second tool calls, that's a 6-second wait turned into a 2-second wait, for one extra line of code. Results come back as a list, **in the order you passed the coroutines in** — not the order they happened to finish.

How it really works

- By default, if one coroutine raises, `gather` raises that same error back to you immediately — but the other coroutines are **not** cancelled; they keep running in the background, their results (and any cost they incur) simply discarded because nothing is waiting to collect them.
- `return_exceptions=True` changes that: `gather` never raises on its own. Instead, the failed coroutine's exception object is placed in the result list at that position, same as a successful result would be, so you check each entry with `isinstance(result, Exception)` afterwards. This is the exact question Doc02's retrying topic asks about a single failed call — does one failure sink the whole batch, or do you want to keep whatever succeeded? — now applied to several calls running together instead of one.
- `gather` always waits for the slowest coroutine, however long that takes — it has no built-in "give up after N seconds." Wrap each call in `asyncio.wait_for(call(), timeout=...)` to cap it individually. This is Doc02's timeout discipline written for the async client instead of `requests` — same idea, same reason: a call with no timeout can hang forever and nothing notices.

- An async HTTP call inside a `gather` still returns a response that can lie the same two ways Doc02 teaches: a `200` with a body that isn't valid JSON, or valid JSON missing the field you actually need. `await` changes nothing about that — check status and shape exactly as you would for a sync call.
- `asyncio.gather(*calls)` needs the star to unpack a list or generator of coroutines into separate arguments — `gather(calls)` (a single list) raises a `TypeError`, because `gather` received one thing instead of several.
- Pairing `gather` with `asyncio.Semaphore(n)` is how you run hundreds of calls "at once" without tripping a `429` — each coroutine acquires the semaphore before calling out and releases it after, so at most `n` are ever in flight.

Situation	What to do	Why
Independent calls; one failure should stop the batch	Plain `gather(a(), b(), c())`	Simple — the first error surfaces right away
One bad call must not lose the good results	`gather(..., return_exceptions=True)`	Every result comes back; check each one for an exception
A slow call must not hang the whole group	`asyncio.wait_for(call(), timeout=...)` per call	`gather` alone always waits for the slowest one
Rate-limited API, many calls	`gather` + `asyncio.Semaphore(n)`	Caps how many are in flight; avoids `429`s
Step 2 needs step 1's answer	Plain `await`, one after another	They aren't independent — nothing to gather

Common mistakes:

- *Mistake:* `asyncio.gather(tasks)` — passing a list without the star. → *Symptom:* a `TypeError` (exact wording depends on Python version), because `gather` got one list instead of several coroutines. → *Fix:* `asyncio.gather(*tasks)`, unpacking the list.
- *Mistake:* assuming a failed call in `gather` cancels the rest, so nothing is "wasted." → *Symptom:* the other coroutines keep running to completion anyway — paid API calls still happen — while your code only ever sees the first exception and never looks at the rest. → *Fix:* decide up front, the way Doc02 makes you decide for a single call: `return_exceptions=True` when partial results matter, and check each one.

Where you'll meet it: Doc09 runs graph nodes in parallel this same way — LangGraph does the gathering for you, but it's the identical idea. Doc11 and Project 4 have several agents work on different parts of one job at once, then combine results — exactly the "keep the good ones" decision above. Doc08's hybrid search runs its keyword and vector searches concurrently with `gather` so

hybrid latency is the max of the two, not the sum. Project 8 — MCPBridge is async throughout and uses this exact pattern for calling several MCP tools at once.

Quick cheat sheet:

- `await asyncio.gather(a(), b(), c())` — time ≈ the slowest call, not the sum.
- Results come back in **input order**, not finish order.
- Building calls in a loop? `gather(*calls)`, with the star.
- `return_exceptions=True` when one failure must not throw away the rest — same question Doc02 asks about a single call.
- Wrap each call in `asyncio.wait_for(..., timeout=...)` — `gather` alone never gives up on its own.

Sync vs. async client: same tool, different way of running

Two doors into the same office: one where you walk in, stand at the counter, and wait until your form is processed before you can do anything else; another where you drop off several forms, get a ticket for each, and go do something else until they're ready. Same office, same forms, two different ways of waiting. Many libraries offer both doors. The OpenAI library has `OpenAI` (sync — each call blocks until the reply arrives) and `AsyncOpenAI` (async — each call is awaited, so the loop can run other tasks meanwhile). `httpx` has `Client` and `AsyncClient`. The method names are nearly identical; the async door just needs `await` in front of every call, and a running event loop underneath it.

How it really works

- FastAPI runs `async def` routes directly on the event loop — a sync client call inside one of those blocks the whole server for every other user during that call. Plain `def` routes are different: FastAPI automatically runs them in a background thread pool, so a sync client there is fine and doesn't block the loop.
- Every timeout rule from Doc02 still applies to the async clients — `AsyncOpenAI(timeout=60.0)`, `httpx.AsyncClient(timeout=10)`. `await` changes how the call waits, not whether it needs a limit on how long it waits.
- `requests` has no async version at all — used inside async code directly, it blocks the loop exactly like any other sync call. `httpx.AsyncClient` is the async-native replacement with the same idea: one `GET`, one `POST`, same status codes, same need to check the JSON shape Doc02 teaches.
- Create the client **once** and reuse it — `AsyncOpenAI()` or `httpx.AsyncClient()` opens network connections that stay open and get reused across calls. Building a new one inside a function called hundreds of times throws that pooling away and is measurably slower under load.
- Converting a function from sync to async is genuinely small: change the import, add `async def`, add `await` before the call, and — only at the very top level — `asyncio.run()`. Nothing about the actual API call changes.

Situation	What to do	Why
Terminal script, one question at a time	Sync `OpenAI()`	No loop needed, fewer lines, easier to debug
Several independent questions to send at once	`AsyncOpenAI()` + `gather`	Real time saved from waiting together
FastAPI route written as `async def`	`AsyncOpenAI()`	A sync call here blocks the whole server
FastAPI route written as plain `def`	Sync `OpenAI()` is fine	FastAPI runs plain `def` routes in a thread pool automatically
Calling a normal HTTP API from async code	`httpx.AsyncClient`, never `requests`	`requests` has no async version and would block

Common mistakes:

- *Mistake:* creating a new `AsyncOpenAI()` or `httpx.AsyncClient()` inside a function called hundreds of times. → *Symptom:* each call opens its own connections instead of reusing pooled ones — slower overall, and connection errors show up under real load. → *Fix:* create the client once (at startup, or once per batch) and pass it to the functions that need it.
- *Mistake:* using the sync `OpenAI()` client inside an `async def` FastAPI route. → *Symptom:* the whole server stops answering every other user for the length of that one call, and nothing raises an error to point at why. → *Fix:* use `AsyncOpenAI`, or if you truly can't, wrap the sync call in `asyncio.to_thread`.

Where you'll meet it: Doc12 and Project 5 call `AsyncOpenAI` inside FastAPI's `async def` routes. Doc09's `ainvoke` / `astream` use an async client underneath. The Real-world exercise below converts a Project 1 function to `AsyncOpenAI` — the exact conversion Project 8 — MCPBridge needs throughout, since its `mcp` SDK client is async-only.

Quick cheat sheet:

- One call at a time, a script → `OpenAI()`. Several at once, or inside `async def` → `AsyncOpenAI()`.
- Doc02's timeout rule applies to async clients exactly the same way.
- Inside async code: `httpx.AsyncClient`, never `requests`.
- Create the client once, reuse it — don't build a new one per call.
- Converting sync → async: new import, `async def`, `await`, one `asyncio.run()` at the bottom.

Common mistakes, and why they're sneaky

A slow leak in a tire doesn't announce itself — the car still drives, just a little worse each day, until one morning it won't start. Most async bugs are exactly that: the code runs, gives an answer, and looks fine, just slower, or with a piece quietly missing — no traceback pointing at the problem. That's what makes this topic worth its own space: the previous three topics each have one or two sharp mistakes; this one is about training yourself to notice the ones that don't announce themselves at all.

How it really works

- A forgotten `await` doesn't crash — it produces a `RuntimeWarning: coroutine '...' was never awaited`, a line that's trivially easy to miss in a busy log, and the coroutine simply never ran.
- A blocking call inside `async def` — the sync `OpenAI` client, `requests.get`, plain `time.sleep()` — freezes the **entire** loop for that call's duration. No exception. Every other task, including ones that were only waiting on a fast network reply, waits behind it too.
- The only reliable way to catch this class of bug is to **time it**: run three copies of a call through `gather` and compare the total against the slowest single call. Close to the slowest → genuinely concurrent. Close to the sum → something inside is blocking.
- `create_task()` without keeping the returned `Task` risks the loop garbage-collecting it mid-flight — the same weak-reference gotcha as the event-loop topic above, worth repeating here because it's one of the quietest failures on this list: no error, the task just doesn't finish.
- Concurrent tasks logging without a per-task id — the gotcha from two topics up — is itself one of these silent bugs: the log looks normal, just wrong, and only a request id in every line lets you untangle which task wrote which line.

Symptom	Likely cause	Fix
`<coroutine object ...>` instead of a value, or `RuntimeWarning: ... was never awaited`	Forgot `await`	Add `await` before the call
`gather` 's total time ≈ the sum, not the slowest	A blocking sync call, or `time.sleep()` , inside `async def`	`await asyncio.sleep()` ; async client; or `asyncio.to_thread`
`RuntimeError: asyncio.run() cannot be called from a running event loop`	`asyncio.run()` called from inside async code	Use `await` instead; `asyncio.run()` once, top-level only
A background task's errors never show up	`create_task()` result not kept or never awaited	Keep the task in a variable/set, `await` it before exit

Common mistakes:

- *Mistake:* trusting "no error" as proof the async code is genuinely running things at the same time. → *Symptom:* everything passes with one user, then the service gets slow the moment ten arrive,

with nothing in the logs pointing at why. → *Fix:* time it — `gather` three copies of a call and check whether the total is close to the slowest one or close to the sum.

- *Mistake:* `asyncio.create_task(job())` with no variable holding the result. → *Symptom:* the task sometimes silently disappears before finishing, and any error it raised is never seen anywhere. → *Fix:* keep the task in a variable or a set, and `await` it (or `gather` it) before the program exits.

Where you'll meet it: Doc14 has a full async debugging drill built on exactly these silent failures. In Project 5 and Project 12, a sync call hidden inside async code is the first thing to check when a service "just hangs." In Doc11, one blocking agent stalls every other agent sharing its process. Doc13 is where timing each step in your logs becomes routine, which is exactly how you notice "parallel" steps that are quietly running one after another.

Quick cheat sheet:

- Most async bugs don't crash — they just run slow, or lose a result, with nothing in the logs.
- Forgot `await`? Watch for `RuntimeWarning: ... was never awaited`.
- `time.sleep()` or a sync client inside `async def` freezes the whole loop, silently.
- Keep a reference to every `create_task()` result, and `await` it.
- When in doubt, time it: `gather` total ≈ slowest call, or something's blocking.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- Python docs — asyncio — the official reference; skim for the shape of it, don't try to memorize every function.
- Real Python — Async IO in Python: A Complete Walkthrough — a longer, clearer explanation.

Practice Exercises

Setup: same venv as before — if it's not active, `cd 08b_async_prereq && source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). Packages for this document: `pip install openai langchain langchain-openai requests` (`langchain` for the Real-world exercise, `requests` for the Failure exercise).

Where your code lives: all of it under `08b_async_prereq/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by level** — two of them share one file, each in its own labelled section — the same convention as Doc01 and Doc02.

The full file layout, all exercises:

```
practice/
├── async_client_conversion_practice.py  Real-world
├── blocking_event_loop_practice.py      Failure
├── coroutine_basics_practice.py         Basic
└── gather_practice.py                   Intermediate + Edge cases
                                          (two sections)
```

Why each script exists:

- `async_client_conversion_practice.py` — the exact conversion Doc12's FastAPI service and Project 8's MCP client both need — a real function, not a toy — plus `async with`, `async for` and LangChain's `ainvoke` / `astream`.
- `blocking_event_loop_practice.py` — the single most common real async bug, caused on purpose here so you recognize its silent symptom later.
- `coroutine_basics_practice.py` — the one fact everything else in this gate assumes: `async def` alone doesn't run anything, only `await` / `asyncio.run()` does — and `create_task()` to start one in the background.
- `gather_practice.py` — the actual speed difference, measured, plus proof that `gather` waits for the slowest task, not the fastest.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure

Basic — your first coroutine

- **What:** an `async def` function that awaits `asyncio.sleep(1)` and returns a value, called with `asyncio.run()`.
- **Why:** you need to see, once, that `async def` alone doesn't run anything — `await` and `asyncio.run()` are what actually make it go.
- **How to code it:** `async def wait_and_return(): await asyncio.sleep(1); return "done"`, then `print(asyncio.run(wait_and_return()))`. Then add a timeout with `asyncio.wait_for(...)`, a check for an already-running loop, and start a coroutine in the background with `asyncio.create_task()`, awaiting it later.
- **Save as:** `practice/coroutine_basics_practice.py`.
- **Used later by:** the Intermediate exercise reuses this exact `asyncio.run()` shape around a timing comparison, and the Real-world exercise reuses it to run a real async client call.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — feel the speed difference

- **What:** run 3 API calls one after another with `await`, timed. Then run the same 3 with `asyncio.gather`, timed. Compare.

- **Why:** the numbers, not the theory, are what actually make "running things at the same time" click — this is the exercise that matters most in this whole document.
- **How to code it:** write 3 `async def` functions that each `await asyncio.sleep(2)`. Time `await f1(); await f2(); await f3()` with `time.perf_counter()`, then time `await asyncio.gather(f1(), f2(), f3())` — compare the two durations directly.
- **Save as:** `practice/gather_practice.py`, under an `# Intermediate` section (this file also holds the Edge cases exercise below, in its own `# Edge cases` section).
- **Builds on:** the Basic exercise's `asyncio.run()` shape — same idea, now with `time.perf_counter()` wrapped around it.
- **Used later by:** any agent making 2+ independent tool calls — this exact pattern is what Doc09's parallel node execution relies on.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — convert a real function to async

- **What:** convert one function from Project 1 (Doc04) to use `AsyncOpenAI` instead of the normal client.
- **Why:** this is the exact conversion Doc12's FastAPI service needs for every route that calls the model — better to do it once here, deliberately, than for the first time under pressure later.
- **How to code it:** replace `from openai import OpenAI` with `from openai import AsyncOpenAI`, make the function `async def`, and `await client.chat.completions.create(...)` instead of calling it directly. Then add a client timeout, call it from an already-async function, open the client with `async with` and read a streamed reply with `async for`, and run a LangChain chain with `ainvoke`, `astream` and `gather`.
- **Save as:** `practice/async_client_conversion_practice.py`.
- **Builds on:** the Basic exercise's `async def` + `asyncio.run()` shape, now wrapping a real API call instead of `asyncio.sleep`.
- **Used later by:** wrapping any agent behind a real API service (Project 5), and Project 8 — MCPBridge, whose `mcp` client is async-only and needs this same conversion throughout.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — proving `gather` actually waits

- **What:** call `asyncio.gather` with one coroutine that finishes fast and one that sleeps for 5 seconds — confirm `gather` waits for the slow one, and that the fast one's result was ready long before that.
- **Why:** it's easy to assume `gather` returns as soon as the *first* task finishes — it doesn't, and you need to have watched it wait for the slowest one, on purpose.
- **How to code it:** one coroutine sleeps 0.1s and returns immediately with a printed timestamp, the other sleeps 5s. Run both through `gather`, and print timestamps before/after to see the fast

one finished long before `gather` actually returned.

- **Save as:** `practice/gather_practice.py`, under an `# Edge cases` section (this file also holds the Intermediate exercise above, in its own `# Intermediate` section).
- **Builds on:** the Intermediate exercise's timing method, in the same file — same `time.perf_counter()` habit, now proving a different fact about `gather`.
- **Used later by:** any parallel-agent design (Doc11) where one agent is much slower than the others — the whole group waits for it.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — accidentally blocking the event loop

- **What:** call a normal, waiting-style function from inside an `async def` function on purpose, without `await` or threading, and watch it kill the "at the same time" behavior you expected.
- **Why:** this is the single most common real async bug, and it fails *silently* — no error, just quietly-worse performance — so you need to have seen it once to recognize it later.
- **How to code it:** inside an `async def`, call `time.sleep(3)` (not `asyncio.sleep`) instead of awaiting something, run it alongside another coroutine with `gather` — the blocking one **first** — and time it. Confirm the total time is now the *sum*, not the max, proving concurrency broke. Then fix it with `asyncio.to_thread(...)`.
- **Save as:** `practice/blocking_event_loop_practice.py`.
- **Builds on:** the timing technique from the Intermediate and Edge cases exercises — same `gather` + `perf_counter()` habit, now used to catch a bug instead of prove a feature.
- **Used later by:** Doc14's async debugging drill, which starts from exactly this broken-on-purpose pattern. This is also the first bug to check when a FastAPI service in Project 5 "just hangs" — a sync `OpenAI` client used where `AsyncOpenAI` was needed.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Expected Behavior

- You can explain, without notes, why `await asyncio.gather(a(), b(), c())` is faster than `await a(); await b(); await c()` for waiting-on-something work — and why it would *not* be faster for heavy-computing work.
- You can spot, in a piece of code, a call that's silently freezing the event loop.

Break-It / Debug Preview

- A coroutine that's defined but never awaited (it silently does nothing — a very common real mistake).
- A normal, waiting-style call hidden inside an async function that kills "at the same time" behavior without raising any error.

- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Event loop basics · `asyncio.gather` vs. one-after-another `await` · sync vs. async client trade-offs · what "blocking the event loop" means and why it's bad.

Move On When

You can correctly explain running-at-the-same-time vs. one-after-another async code, and you've converted at least one real function to async without help. Then move on to 09_langgraph.

Stuck? Ask for **Hint 1** through **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**